

The solution:

State namespaces

Reference: [ADK Documentation - Organizing State with Prefixes](#)

“Prefixes on state keys define their scope and persistence behavior, especially with persistent services”

ADK provides **four state namespaces** with different lifetimes:

Namespace	Prefix	Lifetime	Scope	Example
Temporary	<code>temp:</code>	Current turn only	Discarded after invocation	<code>temp:step = "validating"</code>
Session	(none)	Current conversation	Lost when session ends	<code>topic = "refunds"</code>
User	<code>user:</code>	All sessions for this user	Persists across conversations	<code>user:theme = "dark"</code>
App	<code>app:</code>	Global for all users	Persists for entire app	<code>app:api_url = "..."</code>

Visual timeline

None

graph TD

```
T[temp: state] -->|Discarded after| T1[Turn 1 ends]
S[Session state] -->|Persists until| S1[Session ends]
U[user: state] -->|Persists across| U1[All sessions]
A[app: state] -->|Persists for| A1[All users]
```

```
style T fill:#ffe4e4
style S fill:#fff4e4
style U fill:#e4f4ff
style A fill:#e4ffe4
```

Code example

Python

```
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
```

After running agent

```
session.state["temp:current_step"] = "validation"
session.state["conversation_topic"] = "refunds"
session.state["user:language"] = "Spanish"
session.state["app:version"] = "2.0"
```

```
# ✖ Lost after turn
# ⚠ Lost after session
# ✔ Persists across sessions
# ✔ Global for all users
```


Core concepts

1. Temporary state: **temp:**

Reference: [ADK Documentation - temp: Prefix](#)

From ADK documentation

“**temp:** Prefix (Temporary Invocation State):

- **Scope:** Specific to the current **invocation** (the entire process from an agent receiving user input to generating the final output for that input).
- **Persistence:** **Not persistent.** Discarded after the invocation completes and does not carry over to the next one.
- **Use cases:** Storing intermediate calculations, flags, or data passed between tool calls within a single invocation.”

What is an invocation?

An invocation = **one complete turn** from user input to agent response.

```
None
User sends message
  ↓
[INVOCATION STARTS] ← temp:
state created
  ↓
Agent processes request
  ↓
Agent returns response
  ↓
[INVOCATION ENDS] ← temp:
state DISCARDED
```

Usage

```
Python
from google.adk.agents import
LlmAgent

# Agent using temp: state
agent = LlmAgent(
    instruction="Process request.
Store step: {temp:current_step?
starting}",

    output_key="temp:processing_result"
# Saves to temp: namespace
)

# Turn 1:
# - state["temp:current_step"] =
"validating"
# - After turn ends →
temp:current_step DISCARDED

# Turn 2:
# - state.get("temp:current_step") →
None (was discarded)
```

When to use

- ✓ Intermediate processing steps
- ✓ Temporary flags for current turn only
- ✓ Data that should be discarded after agent completes
- ✗ DON'T use for: Data needed across turns

2. Session state (no prefix)

Reference: [ADK Documentation - No Prefix](#)

From ADK documentation

“No Prefix (Session State):

- **Scope:** Specific to the current session (**id**).
- **Persistence:** Only persists if the **SessionService** is persistent (**Database**, **VertexAI**).
- **Use Cases:** Tracking progress within the current task (e.g., **'current_booking_step'**), temporary flags for this interaction (e.g., **'needs_clarification'**).”

What is a session?

A session = one conversation from start to end.

```
None
User starts conversation
↓
[SESSION STARTS] ← Session state created
↓
Turn 1, Turn 2, Turn 3... (state persists)
↓
User closes/restarts conversation
↓
[SESSION ENDS] ← Session state DISCARDED
```

Usage

```
Python
agent = LlmAgent(
    instruction="Current topic:
{conversation_topic?none}",
    output_key="conversation_topic"
# No prefix = session state
)

# Session 1:
# Turn 1: state["conversation_topic"]
= "refunds"
# Turn 2: state["conversation_topic"]
still "refunds" ✓
# Turn 3: state["conversation_topic"]
still "refunds" ✓
# [Session ends]

# Session 2 (new conversation):
# Turn 1:
state.get("conversation_topic") →
None (new session)
```

When to use

- ✓ Conversation topic tracking
- ✓ Turn counters
- ✓ Shopping cart items (current session only)
- ✗ DON'T use for: Data needed across multiple conversations

3. User state: `user:`

Reference: [ADK Documentation - user: Prefix](#)

From ADK documentation

“`user:` Prefix (User State):

- **Scope:** Tied to the `user_id`, shared across all sessions for that user (within the same `app_name`).
- **Persistence:** Persistent with `Database` or `VertexAI`. (Stored by `InMemory` but lost on restart).
- **Use Cases:** User preferences (e.g., `'user:theme'`), (`'user:theme'`, `'user:name'`).”

Scope

User state persists across all sessions for the same user.

None

User's Session 1:

```
state["user:language"] =  
"Spanish"  
state["user:theme"] = "dark"  
[Session ends]
```

User's Session 2 (days later):

```
state.get("user:language") →  
"Spanish" ✓ (persisted!)  
state.get("user:theme") → "dark"  
✓ (persisted!)
```

Usage

Python

```
agent = LlmAgent(  
    instruction="""  
    You are a helpful assistant.  
  
    User preferences:  
    - Language: {user:language?  
English}  
    - Theme: {user:theme?light}  
  
    Respond in {user:language?  
English}.  
    """)  
)  
  
# First conversation:  
# state["user:language"] = "Spanish"  
# Set once  
  
# Future conversations:  
# Agent automatically uses Spanish  
(reads from user: state)
```

When to use

- ✓ Language preferences
- ✓ Display theme (dark/light)
- ✓ Subscription tier
- ✓ User profile information
- ✗ DON'T use for: Data specific to current conversation

4. App state: **app:**

Reference: [ADK Documentation - app: Prefix](#)

From ADK documentation

“**app:** Prefix (App State):

- **Scope:** Tied to the **app_name**, shared across all users and sessions for that application.
- **Persistence:** Persistent with **Database** or **VertexAI**. (Stored by **InMemory** but lost on restart).
- **Use Cases:** Global settings (e.g., `'app:api_endpoint'`), shared templates.”

Scope

App state is **global**—all users see the same values.

None

User A, Session 1:
state["app:api_url"] = "https://api.example.com"

User B, Session 1:
state.get("app:api_url") →
"https://api.example.com" ✓
(shared!)

User C, Session 5:
state.get("app:api_url") →
"https://api.example.com" ✓ (still
shared!)

Usage

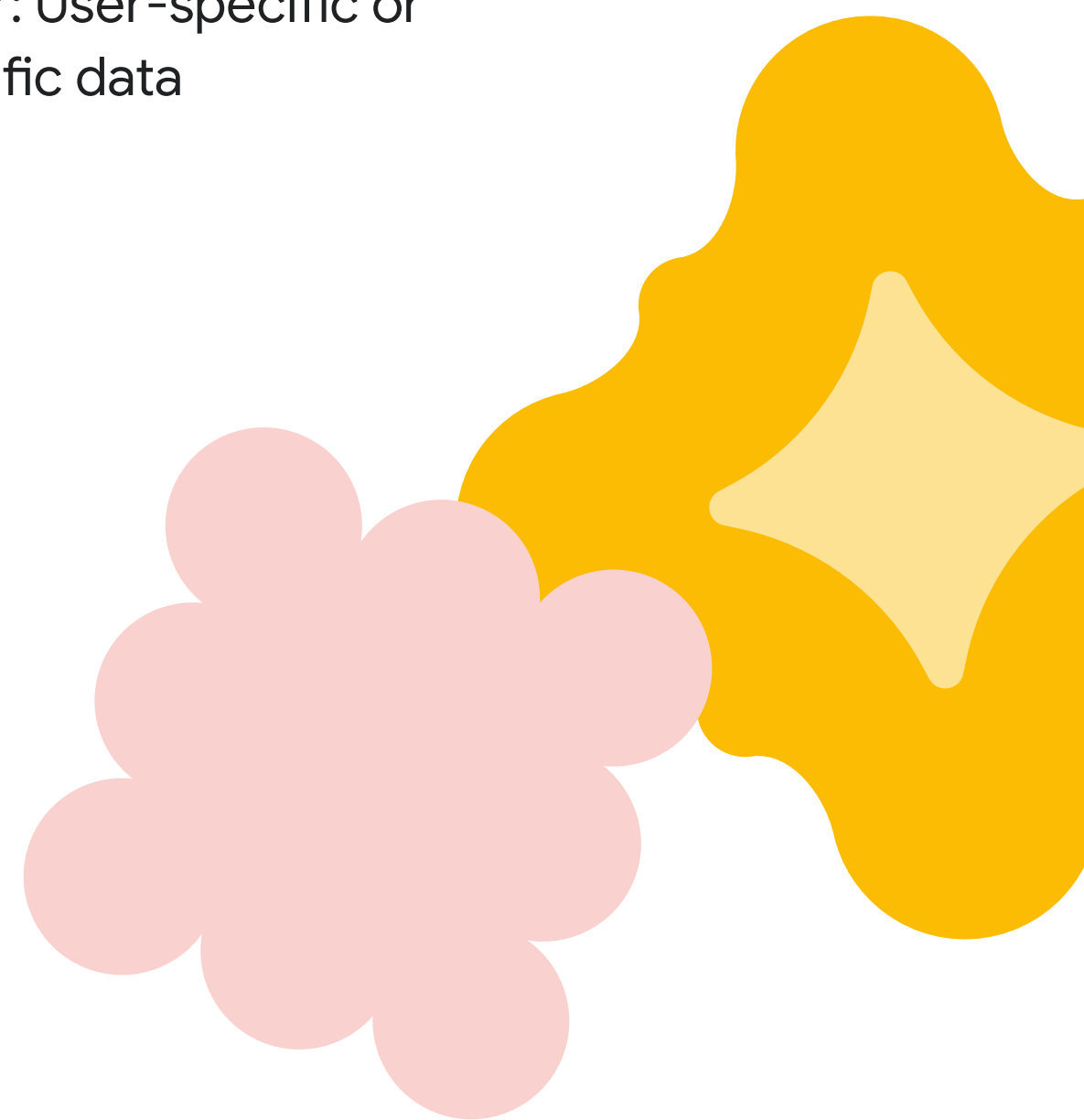
```
Python
agent = LlmAgent(
    instruction="""
    You are version {app:version?
    1.0}.

    API endpoint: {app:api_url?not
    set}
    Feature flags: {app:features?
    basic}
    """
)

# Set once, applies to all users:
# state["app:version"] = "2.0"
# state["app:api_url"] = "https://
api.example.com"
```

When to use

- ✓ API endpoints
- ✓ Feature flags
- ✓ Global configuration
- ✓ Application version
- ✗ DON'T use for: User-specific or session-specific data



Hands-on example

Building an agent that demonstrates all four namespaces

What you'll build: An agent that uses all four state namespaces to show their different persistence behaviors.

Step 1: Create the project

```
Shell
adk create namespace_demo
cd namespace_demo
```

Step 2: Write the agent

Replace the contents of `agent.py` with:

```
Python
"""
Namespace Demo - Shows all four state namespaces
Demonstrates temp:, session, user:,
and app: persistence scopes.

Reference: https://google.github.io/
adk-docs/sessions/state.md
"""

from google.adk.agents import LlmAgent

# Create agent that uses all four namespaces
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='namespace_demo',

    instruction="""
    You are a demo assistant showing
    state namespaces.
```

```
    """
    """
    App State (global for all users)
    """
    App name: {app:name?Namespace
    Demo}
    App version: {app:version?1.0}

    """
    User State (persists across
    sessions) """
    User preference: {user:theme?not
    set}

    """
    Session State (persists this
    conversation) """
    Conversation topic: {topic?not
    set}

    """
    Temp State (current turn only)
    """
    Current step: {temp:step?not set}

    Respond with a friendly message
    showing these namespace values.
    """
    output_key="response"
)
```


Step 3: Test with Runner

Create `test_namespaces.py`:

Python

"""

Test state namespaces to see persistence differences.

Run with: `python test_namespaces.py`

"""

```
from agent import root_agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part
```

Setup

```
session_service = InMemorySessionService()
session = session_service.create_session(
    app_name="namespace_demo_app",
    user_id="user1",
    session_id="session1"
)
```

```
runner = Runner(
    agent=root_agent,
    app_name="namespace_demo_app",
    session_service=session_service
)
```

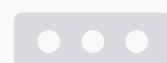
Set all four namespace types

```
print("=== Setting state in all namespaces ===")
session.state["app:name"] = "Namespace Demo"
session.state["app:version"] = "2.0"
session.state["user:theme"] = "dark"
session.state["topic"] = "state management"
session.state["temp:step"] = "initialization"
```

```
print(f"State before run: {session.state}\n")
```

Run agent

```
print("=== Running agent (Turn 1) ===")
result = runner.run(
    user_id="user1",
    session_id="session1",
    new_message=Content(parts=[Part(text="Show me the namespace values")])
)
```




```

# Show response
for event in result:
    if event.is_final_response():
        print(f"Agent response:\n{event.content.parts[0].text}\n")

# Check state after turn
print("=== State after Turn 1 ===")
print(f"Full state: {session.state}")
print(f"temp:step: {session.state.get('temp:step')}") # Should be GONE
print(f"topic: {session.state.get('topic')}") # Should persist
print(f"user:theme: {session.state.get('user:theme')}") # Should persist
print(f"app:version: {session.state.get('app:version')}") # Should persist

print("\n=== Simulating Turn 2 (same session) ===")
result2 = runner.run(
    user_id="user1",
    session_id="session1",
    new_message=Content(parts=[Part(text="Check state again")])
)

for event in result2:
    if event.is_final_response():
        print(f"Agent response:\n{event.content.parts[0].text}\n")

print("=== State after Turn 2 ===")
print(f"Full state: {session.state}")
print(f"temp:step: {session.state.get('temp:step')}") # Still GONE
print(f"topic: {session.state.get('topic')}") # Still here (session state)
print(f"user:theme: {session.state.get('user:theme')}") # Still here (user state)

# Simulate new session
print("\n=== Simulating NEW Session (session2) ===")
session2 = session_service.create_session(
    app_name="namespace_demo_app",
    user_id="user1", # Same user
    session_id="session2" # Different session
)

print(f"New session state: {session2.state}")
print(f"topic: {session2.state.get('topic')}") # Should be GONE (session-scoped)
print(f"user:theme: {session2.state.get('user:theme')}") # Should PERSIST (user-scoped)
print(f"app:version: {session2.state.get('app:version')}") # Should PERSIST (app-scoped)

```



Run it:

```
Shell
python test_namespaces.py
```

Expected output:

```
None
=== Setting state in all namespaces
===
State before run: {'app:name':
'Namespace Demo', 'app:version':
'2.0', 'user:theme': 'dark', 'topic':
'state management', 'temp:step':
'initialization'}

=== Running agent (Turn 1) ===
Agent response:
Hello! Here are the namespace values:
- App: Namespace Demo (v2.0)
- User theme: dark
- Topic: state management
- Current step: initialization

=== State after Turn 1 ===
Full state: {'app:name': 'Namespace
Demo', 'app:version': '2.0',
'user:theme': 'dark', 'topic': 'state
management', 'response': '...'}
temp:step: None ← GONE after turn!
topic: state management ← Persisted
(session)
user:theme: dark ← Persisted (user)
app:version: 2.0 ← Persisted (app)

=== Simulating Turn 2 (same session)
===
Agent response:
Hello again! Values are:
- App: Namespace Demo (v2.0)
- User theme: dark
- Topic: state management
- Current step: not set ← temp: was
discarded
```

```
=== State after Turn 2 ===
Full state: {same as after Turn 1}
temp:step: None ← Still GONE
topic: state management ← Still here
user:theme: dark ← Still here

=== Simulating NEW Session (session2)
===
New session state: {'app:name':
'Namespace Demo', 'app:version':
'2.0', 'user:theme': 'dark'}
topic: None ← GONE (was session-
scoped)
user:theme: dark ← PERSISTED across
sessions!
app:version: 2.0 ← PERSISTED
globally!
```

What to notice

- ✓ `temp:step` disappeared after turn 1 (temporary)
- ✓ `topic` persisted through turn 2 but lost in new session (session-scoped)
- ✓ `user:theme` persisted across sessions for same user (user-scoped)
- ✓ `app:version` persisted globally (app-scoped)

Key takeaways

Four namespaces with different lifetimes:

```
Python
state["temp:var"]      # ❌ Discarded after turn
state["var"]           # ⚠ Discarded when session ends
state["user:var"]      # ✅ Persists across sessions (same user)
state["app:var"]       # ✅ Persists globally (all users)
```

Decision tree:

```
None
How long should the data persist?

├─ Only need RIGHT NOW (current turn)?
│   └─ Use temp: namespace
│       Example:
temp:processing_step,
temp:validation_status
├─ Need this CONVERSATION only?
│   └─ Use session state (no prefix)
│       Example: conversation_topic,
turn_count, shopping_cart
└─ Need across ALL conversations for this USER?
    └─ Use user: namespace
        Example: user:language,
user:theme, user:tier
└─ Need GLOBALLY for all users?
    └─ Use app: namespace
        Example: app:api_url,
app:version, app:feature_flags
```

Best practice: Use narrowest scope needed

- Don't persist data longer than necessary
- **temp:** for intermediate steps
- Session for conversation context
- **user:** for preferences
- **app:** for global config

All namespaces work with templating:

```
None
instruction="""
App: {app:name?MyApp}
User: {user:language?English}
Topic: {topic?none}
Step: {temp:step?start}
"""
```

What's next:

You've completed lesson 4!

Next, you'll learn about tools in [Lesson 5: Add Capabilities with Tools](#), which includes memory search and other advanced capabilities.